Práctica 6 - Backtracking

1)CUADRADO NUMÉRICO

a) Implementar en Java dicho algoritmo (CuadradoNumerico.java) de tal forma que calcule para una entrada de tablero dada una solución de la forma más eficiente posible.

b) Implementar en Java dicho algoritmo (CuadradoNumericoTodas.java) de tal forma que calcule para una entrada de tablero dada todas las soluciones posibles.

c) Razonar la Complejidad Temporal. ¿Cree que se podría resolver este problema de forma óptima utilizando algunas de las técnicas anteriores vistas durante el curso?

Caso de prueba	Tiempo para la primera solución (ms)	Tiempo para todas las soluciones (ms)	Número de soluciones encontradas
Test00	FdF	FdF	1
Test01	FdF	62	13
Test02	FdF	FdF	1
Test03	1227	2006	3
Test04	16031	21903	2
Test05	488	5482	11
Test06	127	52252	237
Test07	6615	FdT	-

Al tratarse de un caso de backtracking, sabemos que la complejidad de este viene dada por el número de nodos generados, así como el tiempo invertido en cada uno de estos.

En nuestro caso, al rellenar cada casilla un total de diez veces (de 0 a 9), la complejidad sería del número total a casillas por rellenar (las ‘?’) multiplicado por esto.

Por ende, es evidente que a mayor número de interrogantes (que no es lo mismo que mayor tamaño de matriz, aunque pueden ir de la mano) mayores serán los tiempos obtenidos.

Respecto a una puntualización de mi código y los test subidos al campus, en los test de un solo caso dos tests en los que el cuadrado matemático resultante difiere del esperado, pero aún así, al menos a mi modo de verlo, los que se obtienen con mi método de backtracking son válidos.

Y de una forma similar, aunque en este caso no estoy tan seguro de que sea correcto, hay ciertos casos en los tests de Todas las soluciones en los que el número de soluciones totales encontradas no coinciden, no sé si es que hay un margen de error, visto que las soluciones que obtengo yo difieren en algunos casos, o no, pero veía importante destacarlo.